Matrices

```
>>> from sympy import *
>>> init_printing(use_unicode=True)
```

To make a matrix in SymPy, use the `Matrix` object. A matrix is constructed by providing a list of row vectors that make up the matrix. For example, to construct the matrix

$$\begin{bmatrix} 1 & -1 \\ 3 & 4 \\ 0 & 2 \end{bmatrix}$$

use

```
>>> Matrix([[1, -1], [3, 4], [0, 2]])
⎡1  -1⎤
⎢     ⎥
⎢3  4 ⎥
⎢     ⎥
⎣0  2 ⎦
```

To make it easy to make column vectors, a list of elements is considered to be a column vector.

```
>>> Matrix([1, 2, 3])
⎡1⎤
⎢ ⎥
⎢2⎥
⎢ ⎥
⎣3⎦
```

Matrices are manipulated just like any other object in SymPy or Python.

```
>>> M = Matrix([[1, 2, 3], [3, 2, 1]])
>>> N = Matrix([0, 1, 1])
>>> M*N
⎡5⎤
⎢ ⎥
⎣3⎦
```

One important thing to note about SymPy matrices is that, unlike every other object in SymPy, they are mutable. This means that they can be modified in place, as we will see below. The downside to this is that `Matrix` cannot be used in places that require immutability, such as inside

Skip to content

essions or as keys to dictionaries. If you need an immutable version of `Matrix`, `rix`.

Basic Operations

Here are some basic operations on `Matrix`.

Shape

To get the shape of a matrix, use `shape()` function.

```
>>> from sympy import shape
>>> M = Matrix([[1, 2, 3], [-2, 0, 4]])
>>> M
⎡1   2  3⎤
⎢        ⎥
⎣-2  0  4⎦
>>> shape(M)
(2, 3)
```

Accessing Rows and Columns

To get an individual row or column of a matrix, use `row` or `col`. For example, `M.row(0)` will get the first row. `M.col(-1)` will get the last column.

```
>>> M.row(0)
[1  2  3]
>>> M.col(-1)
⎡3⎤
⎢ ⎥
⎣4⎦
```

Deleting and Inserting Rows and Columns

To delete a row or column, use `row_del` or `col_del`. These operations will modify the Matrix **in place**.

```
>>> M.col_del(0)
>>> M
⎡2  3⎤
⎢    ⎥
⎣0  4⎦
>>> M.row_del(1)
>>> M
[2  3]
```

To insert rows or columns, use `row_insert` or `col_insert`. These operations **do not** operate in place.

Skip to content

```
>>> M
[2  3]
>>> M = M.row_insert(1, Matrix([[0, 4]]))
>>> M
⎡2  3⎤
⎢    ⎥
⎣0  4⎦
>>> M = M.col_insert(0, Matrix([1, -2]))
>>> M
⎡1   2  3⎤
⎢        ⎥
⎣-2  0  4⎦
```

Unless explicitly stated, the methods mentioned below do not operate in place. In general, a method that does not operate in place will return a new `Matrix` and a method that does operate in place will return `None`.

Basic Methods

As noted above, simple operations like addition, multiplication and power are done just by using `+`, `*`, and `**`. To find the inverse of a matrix, just raise it to the `-1` power.

```
>>> M = Matrix([[1, 3], [-2, 3]])
>>> N = Matrix([[0, 3], [0, 7]])
>>> M + N
⎡1   6 ⎤
⎢      ⎥
⎣-2  10⎦
>>> M*N
⎡0  24⎤
⎢     ⎥
⎣0  15⎦
>>> 3*M
⎡3   9⎤
⎢     ⎥
⎣-6  9⎦
>>> M**2
⎡-5  12⎤
⎢      ⎥
⎣-8  3 ⎦
>>> M**-1
⎡1/3  -1/3⎤
⎢         ⎥
⎣2/9  1/9 ⎦
>>> N**-1
Traceback (most recent call last):
...
NonInvertibleMatrixError: Matrix det == 0; not invertible.
```

To take the transpose of a Matrix, use `T`.

Skip to content

```
>>> M = Matrix([[1, 2, 3], [4, 5, 6]])
>>> M
⎡1  2  3⎤
⎢       ⎥
⎣4  5  6⎦
>>> M.T
⎡1  4⎤
⎢    ⎥
⎢2  5⎥
⎢    ⎥
⎣3  6⎦
```

Matrix Constructors

Several constructors exist for creating common matrices. To create an identity matrix, use `eye`. `eye(n)` will create an $n \times n$ identity matrix.

```
>>> eye(3)
⎡1  0  0⎤
⎢       ⎥
⎢0  1  0⎥
⎢       ⎥
⎣0  0  1⎦
>>> eye(4)
⎡1  0  0  0⎤
⎢          ⎥
⎢0  1  0  0⎥
⎢          ⎥
⎢0  0  1  0⎥
⎢          ⎥
⎣0  0  0  1⎦
```

To create a matrix of all zeros, use `zeros`. `zeros(n, m)` creates an $n \times m$ matrix of 0s.

```
>>> zeros(2, 3)
⎡0  0  0⎤
⎢       ⎥
⎣0  0  0⎦
```

Similarly, `ones` creates a matrix of ones.

```
>>> ones(3, 2)
⎡1  1⎤
⎢    ⎥
⎢1  1⎥
⎢    ⎥
⎣1  1⎦
```

Skip to content

l matrices, use `diag`. The arguments to `diag` can be either numbers or
er is interpreted as a 1×1 matrix. The matrices are stacked diagonally. The
remaining elements are filled with 0s.

```
>>> diag(1, 2, 3)
⎡1  0  0⎤
⎢       ⎥
⎢0  2  0⎥
⎢       ⎥
⎣0  0  3⎦
>>> diag(-1, ones(2, 2), Matrix([5, 7, 5]))
⎡-1  0  0  0⎤
⎢           ⎥
⎢0   1  1  0⎥
⎢           ⎥
⎢0   1  1  0⎥
⎢           ⎥
⎢0   0  0  5⎥
⎢           ⎥
⎢0   0  0  7⎥
⎢           ⎥
⎣0   0  0  5⎦
```

Advanced Methods

Determinant

To compute the determinant of a matrix, use `det`.

```
>>> M = Matrix([[1, 0, 1], [2, -1, 3], [4, 3, 2]])
>>> M
⎡1  0   1⎤
⎢        ⎥
⎢2  -1  3⎥
⎢        ⎥
⎣4  3   2⎦
>>> M.det()
-1
```

RREF

To put a matrix into reduced row echelon form, use `rref`. `rref` returns a tuple of two elements. The first is the reduced row echelon form, and the second is a tuple of indices of the pivot columns.

```
>>> M = Matrix([[1, 0, 1, 3], [2, 3, 4, 7], [-1, -3, -3, -4]])
>>> M
⎡1   0   1   3 ⎤
⎢              ⎥
⎢2   3   4   7 ⎥
⎢              ⎥
⎣-1  -3  -3  -4⎦
>>> M.rref()
⎛⎡1  0   1    3 ⎤        ⎞
⎜⎢               ⎥        ⎟
⎜⎢0  1  1/3  1/3⎥, (0, 1)⎟
⎜⎢               ⎥        ⎟
⎝⎣0  0   0    0 ⎦        ⎠
```

Skip to content

> **Note**
>
> The first element of the tuple returned by `rref` is of type `Matrix`. The second is of type `tuple`.

Nullspace

To find the nullspace of a matrix, use `nullspace`. `nullspace` returns a `list` of column vectors that span the nullspace of the matrix.

```
>>> M = Matrix([[1, 2, 3, 0, 0], [4, 10, 0, 0, 1]])
>>> M
⎡1  2   3  0  0⎤
⎢              ⎥
⎣4  10  0  0  1⎦
>>> M.nullspace()
⎡⎡-15⎤  ⎡0⎤  ⎡ 1  ⎤⎤
⎢⎢   ⎥  ⎢ ⎥  ⎢    ⎥⎥
⎢⎢ 6  ⎥  ⎢0⎥  ⎢-1/2⎥⎥
⎢⎢    ⎥  ⎢ ⎥  ⎢    ⎥⎥
⎢⎢ 1  ⎥, ⎢0⎥, ⎢ 0  ⎥⎥
⎢⎢    ⎥  ⎢ ⎥  ⎢    ⎥⎥
⎢⎢ 0  ⎥  ⎢1⎥  ⎢ 0  ⎥⎥
⎢⎢    ⎥  ⎢ ⎥  ⎢    ⎥⎥
⎣⎣ 0  ⎦  ⎣0⎦  ⎣ 1  ⎦⎦
```

Columnspace

To find the columnspace of a matrix, use `columnspace`. `columnspace` returns a `list` of column vectors that span the columnspace of the matrix.

```
>>> M = Matrix([[1, 1, 2], [2 ,1 , 3], [3 , 1, 4]])
>>> M
⎡1  1  2⎤
⎢       ⎥
⎢2  1  3⎥
⎢       ⎥
⎣3  1  4⎦
>>> M.columnspace()
⎡⎡1⎤  ⎡1⎤⎤
⎢⎢ ⎥  ⎢ ⎥⎥
⎢⎢2⎥, ⎢1⎥⎥
⎢⎢ ⎥  ⎢ ⎥⎥
⎣⎣3⎦  ⎣1⎦⎦
```

Eigenvalues, Eigenvectors, and Diagonalization

To find the eigenvalues of a matrix, use `eigenvals`. `eigenvals` returns a dictionary of `eigenvalue: algebraic_multiplicity` pairs (similar to the output of roots).

Skip to content

```
>>> M = Matrix([[3, -2,  4, -2], [5,  3, -3, -2], [5, -2,  2, -2], [5, -2, -3,
>>> M
⎡3  -2  4   -2⎤
⎢             ⎥
⎢5  3   -3  -2⎥
⎢             ⎥
⎢5  -2  2   -2⎥
⎢             ⎥
⎣5  -2  -3  3 ⎦
>>> M.eigenvals()
{-2: 1, 3: 1, 5: 2}
```

This means that `M` has eigenvalues -2, 3, and 5, and that the eigenvalues -2 and 3 have algebraic multiplicity 1 and that the eigenvalue 5 has algebraic multiplicity 2.

To find the eigenvectors of a matrix, use `eigenvects`. `eigenvects` returns a list of tuples of the form `(eigenvalue, algebraic_multiplicity, [eigenvectors])`.

```
>>> M.eigenvects()
⎡⎛          ⎡⎡0⎤⎤⎞  ⎛      ⎡⎡1⎤⎤⎞  ⎛      ⎡⎡1⎤  ⎡0 ⎤⎤⎞⎤
⎢⎜          ⎢⎢ ⎥⎥⎟  ⎜      ⎢⎢ ⎥⎥⎟  ⎜      ⎢⎢ ⎥  ⎢  ⎥⎥⎟⎥
⎢⎜          ⎢⎢1⎥⎥⎟  ⎜      ⎢⎢1⎥⎥⎟  ⎜      ⎢⎢1⎥  ⎢-1⎥⎥⎟⎥
⎢⎜-2, 1, ⎢⎢ ⎥⎥⎟, ⎜3, 1, ⎢⎢ ⎥⎥⎟, ⎜5, 2, ⎢⎢ ⎥, ⎢  ⎥⎥⎟⎥
⎢⎜          ⎢⎢1⎥⎥⎟  ⎜      ⎢⎢1⎥⎥⎟  ⎜      ⎢⎢ ⎥  ⎢0 ⎥⎥⎟⎥
⎢⎜          ⎢⎢ ⎥⎥⎟  ⎜      ⎢⎢1⎥⎥⎟  ⎜      ⎢⎣0⎦  ⎢  ⎥⎥⎟⎥
⎣⎝          ⎣⎣1⎦⎦⎠  ⎝      ⎣⎣1⎦⎦⎠  ⎝      ⎣    ⎣1 ⎦⎦⎠⎦
```

This shows us that, for example, the eigenvalue 5 also has geometric multiplicity 2, because it has two eigenvectors. Because the algebraic and geometric multiplicities are the same for all the eigenvalues, `M` is diagonalizable.

To diagonalize a matrix, use `diagonalize`. `diagonalize` returns a tuple (P, D), where D is diagonal and $M = PDP^{-1}$.

Skip to content

```
>>> P, D = M.diagonalize()
>>> P
⎡0  1  1  0 ⎤
⎢            ⎥
⎢1  1  1  -1⎥
⎢            ⎥
⎢1  1  1  0 ⎥
⎢            ⎥
⎣1  1  0  1 ⎦
>>> D
⎡-2  0  0  0⎤
⎢           ⎥
⎢0   3  0  0⎥
⎢           ⎥
⎢0   0  5  0⎥
⎢           ⎥
⎣0   0  0  5⎦
>>> P*D*P**-1
⎡3  -2  4   -2⎤
⎢             ⎥
⎢5  3   -3  -2⎥
⎢             ⎥
⎢5  -2  2   -2⎥
⎢             ⎥
⎣5  -2  -3  3 ⎦
>>> P*D*P**-1 == M
True
```

Quick Tip

`lambda` is a reserved keyword in Python, so to create a Symbol called λ, while using the same names for SymPy Symbols and Python variables, use `lamda` (without the `b`). It will still pretty print as λ.

Note that since `eigenvects` also includes the eigenvalues, you should use it instead of `eigenvals` if you also want the eigenvectors. However, as computing the eigenvectors may often be costly, `eigenvals` should be preferred if you only wish to find the eigenvalues.

If all you want is the characteristic polynomial, use `charpoly`. This is more efficient than `eigenvals`, because sometimes symbolic roots can be expensive to calculate.

```
>>> lamda = symbols('lamda')
>>> p = M.charpoly(lamda)
>>> factor(p.as_expr())
      2
(λ - 5) ·(λ - 3)·(λ + 2)
```

Skip to content

Possible Issues

Zero Testing

If your matrix operations are failing or returning wrong answers, the common reasons would likely be from zero testing. If there is an expression not properly zero-tested, it can possibly bring issues in finding pivots for gaussian elimination, or deciding whether the matrix is inversible, or any high level functions which relies on the prior procedures.

Currently, the SymPy's default method of zero testing `_iszero` is only guaranteed to be accurate in some limited domain of numerics and symbols, and any complicated expressions beyond its decidability are treated as `None`, which behaves similarly to logical `False`.

The list of methods using zero testing procedures are as follows:

`echelon_form` , `is_echelon` , `rank` , `rref` , `nullspace` , `eigenvects` , `inverse_ADJ` , `inverse_GE` , `inverse_LU` , `LUdecomposition` , `LUdecomposition_Simple` , `LUsolve`

They have property `iszerofunc` opened up for user to specify zero testing method, which can accept any function with single input and boolean output, while being defaulted with `_iszero`.

Here is an example of solving an issue caused by undertested zero. While the output for this particular matrix has since been improved, the technique below is still of interest. [1] [2] [3]

```
>>> from sympy import *
>>> q = Symbol("q", positive = True)
>>> m = Matrix([
... [-2*cosh(q/3),      exp(-q),            1],
... [      exp(q), -2*cosh(q/3),            1],
... [           1,            1, -2*cosh(q/3)]])
>>> m.nullspace()
[]
```

You can trace down which expression is being underevaluated, by injecting a custom zero test with warnings enabled.

```
>>> import warnings
>>>
>>> def my_iszero(x):
...     result = x.is_zero
...
...     # Warnings if evaluated into None
...     if result is None:
...         warnings.warn("Zero testing of {} evaluated into None".format(x))
...     return result
...
>>> m.nullspace(iszerofunc=my_iszero)
__main__:9: UserWarning: Zero testing of 4*cosh(q/3)**2 - 1 evaluated into None
  ''serWarning: Zero testing of (-exp(q) - 2*cosh(q/3))*(-2*cosh(q/3) -
  serWarning: Zero testing of 2*exp(q)*cosh(q/3) - 16*cosh(q/3)**4 +
  serWarning: Zero testing of -(4*cosh(q/3)**2 - 1)*exp(-q) - 2*cosh(
[]
```

Skip to content

In this case, `(-exp(q) - 2*cosh(q/3))*(-2*cosh(q/3) - exp(-q)) - (4*cosh(q/3)**2 - 1)**2` should yield zero, but the zero testing had failed to catch. possibly meaning that a stronger zero test should be introduced. For this specific example, rewriting to exponentials and applying simplify would make zero test stronger for hyperbolics, while being harmless to other polynomials or transcendental functions.

```
>>> def my_iszero(x):
...     result = x.rewrite(exp).simplify().is_zero
...
...     # Warnings if evaluated into None
...     if result is None:
...         warnings.warn("Zero testing of {} evaluated into None".format(x))
...     return result
...
>>> m.nullspace(iszerofunc=my_iszero)
__main__:9: UserWarning: Zero testing of -2*cosh(q/3) - exp(-q) evaluated into N
⎡⎡  ⎛   q         ⎛q⎞⎞  -q         2⎛q⎞    ⎤⎤
⎢⎢- ⎝- ℯ  - 2⋅cosh⎜─⎟⎟⋅ℯ   + 4⋅cosh ⎜─⎟ - 1⎥⎥
⎢⎢  ⎝              ⎝3⎠⎠              ⎝3⎠    ⎥⎥
⎢⎢──────────────────────────────────────────⎥⎥
⎢⎢                ⎛      2⎛q⎞    ⎞   ⎛q⎞   ⎥⎥
⎢⎢        2⋅      ⎜4⋅cosh ⎜─⎟ - 1⎟⋅cosh⎜─⎟   ⎥⎥
⎢⎢                ⎝       ⎝3⎠    ⎠   ⎝3⎠   ⎥⎥
⎢⎢                                         ⎥⎥
⎢⎢                ⎛   q         ⎛q⎞⎞       ⎥⎥
⎢⎢              -⎝- ℯ  - 2⋅cosh⎜─⎟⎠       ⎥⎥
⎢⎢                ⎝             ⎝3⎠⎠       ⎥⎥
⎢⎢              ──────────────────────     ⎥⎥
⎢⎢                      2⎛q⎞               ⎥⎥
⎢⎢                 4⋅cosh ⎜─⎟ - 1          ⎥⎥
⎢⎢                        ⎝3⎠              ⎥⎥
⎢⎢                                         ⎥⎥
⎣⎣                   1                     ⎦⎦
```

You can clearly see `nullspace` returning proper result, after injecting an alternative zero test.

Note that this approach is only valid for some limited cases of matrices containing only numerics, hyperbolics, and exponentials. For other matrices, you should use different method opted for their domains.

Possible suggestions would be either taking advantage of rewriting and simplifying, with tradeoff of speed [4] , or using random numeric testing, with tradeoff of accuracy [5] .

If you wonder why there is no generic algorithm for zero testing that can work with any symbolic entities, it's because of the constant problem stating that zero testing is undecidable [6] , and not only the SymPy, but also other computer algebra systems [7] [8] would face the same fundamental issue.

However, discovery of any zero test failings can provide some good examples to improve SymPy,

Skip to content

ountered one, you can report the issue to SymPy issue tracker [9] to get detailed
mmunity.

Footnotes

[1] Inspired by https://gitter.im/sympy/sympy?at=5b7c3e8ee5b40332abdb206c

[2] Discovered from https://github.com/sympy/sympy/issues/15141

[3] Improved by https://github.com/sympy/sympy/pull/19548

[4] Suggested from https://github.com/sympy/sympy/issues/10120

[5] Suggested from https://github.com/sympy/sympy/issues/10279

[6] https://en.wikipedia.org/wiki/Constant_problem

[7] How mathematica tests zero https://reference.wolfram.com/language/ref/PossibleZeroQ.html

[8] How matlab tests zero https://web.archive.org/web/20200307091449/https://www.mathworks.com/help/symbolic/mupad_ref/iszero.html

[9] https://github.com/sympy/sympy/issues



Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024